

```

    } else {
        if (u < x) a=u; else b=u;
        if (fu <= fw || w == x) {
            v=w;
            w=u;
            fv=fw;
            fw=fu;
        } else if (fu <= fv || v == x || v == w) {
            v=u;
            fv=fu;
        }
    }
}
throw("Too many iterations in brent");
};

```

Done with housekeeping. Back for
another iteration.

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 5.[1]
 Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §8.2.

10.4 One-Dimensional Search with First Derivatives

Here we want to accomplish precisely the same goal as in the previous section, namely to isolate a functional minimum that is bracketed by the triplet of abscissas (a, b, c) , but utilizing an additional capability to compute the function's first derivative as well as its value.

In principle, we might simply search for a zero of the derivative, ignoring the function value information, using a root finder like `rtflsp` or `zbrent` (§9.2 – §9.3). It doesn't take long to reject *that* idea: How do we distinguish maxima from minima? Where do we go from initial conditions where the derivatives on one or both of the outer bracketing points indicate that “downhill” is in the direction *out* of the bracketed interval?

We don't want to give up our strategy of maintaining a rigorous bracket on the minimum at all times. The only way to keep such a bracket is to update it using function (not derivative) information, with the central point in the bracketing triplet always that with the lowest function value. Therefore, the role of the derivatives can only be to help us choose new trial points within the bracket.

One school of thought is to “use everything you've got”: Compute a polynomial of relatively high order (cubic or above) that agrees with some number of previous function and derivative evaluations. For example, there is a unique cubic that agrees with function and derivative at two points, and one can jump to the interpolated minimum of that cubic (if there is a minimum within the bracket). Suggested by Davidon and others, formulas for this tactic are given in [1].

We like to be more conservative than this. Once superlinear convergence sets in, it hardly matters whether its order is moderately lower or higher. In practical

problems that we have met, most function evaluations are spent in getting globally close enough to the minimum for superlinear convergence to commence. So we are more worried about all the funny “stiff” things that high-order polynomials can do (cf. Figure 3.0.1(b)), and about their sensitivities to roundoff error.

This leads us to use derivative information only as follows: The sign of the derivative at the central point of the bracketing triplet (a, b, c) indicates uniquely whether the next test point should be taken in the interval (a, b) or in the interval (b, c) . The value of this derivative and of the derivative at the second-best-so-far point are extrapolated to zero by the secant method (inverse linear interpolation), which by itself is superlinear of order 1.618. (The golden mean again: See [1], p. 57.) We impose the same sort of restrictions on this new trial point as in Brent’s method. If the trial point must be rejected, we *bisect* the interval under scrutiny.

Yes, we are fuddy-duddies when it comes to making flamboyant use of derivative information in one-dimensional minimization. But we have met too many functions whose computed “derivatives” *don’t* integrate up to the function value and *don’t* accurately point the way to the minimum, usually because of roundoff errors, sometimes because of truncation error in the method of derivative evaluation.

You will see that the following routine is closely modeled on Brent in the previous section. One difference is in the input to the routine. Whereas Brent takes either a function or functor argument, Dbrent takes only a functor. The functor returns not only the function value, by overloading operator(), but also the derivative as the member function df. For example, here’s how you would code the function x^2 :

```
struct Functd {                               Name Functd is arbitrary.
    Doub operator() (const Doub x) {
        return x*x;
    }
    Doub df(const Doub x) {
        return 2.0*x;
    }
};
```

To invoke Dbrent, you need statements like the following:

```
Dbrent dbrent;
Functd f;
dbrent.bracket(a,b,f);
xmin=dbrent.minimize(f);
```

The value of the function at the minimum is available in dbrent.fmin as usual. Here is the routine:

```
mins.h struct Dbrent : Bracketmethod {
Brent's method to find a minimum, modified to use derivatives.
    Doub xmin,fmin;
    const Doub tol;
    Dbrent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &functd)
    Given a functor functd that computes a function and also its derivative function df, and
    given a bracketing triplet of abscissas ax, bx, cx [such that bx is between ax and cx, and
    f(bx) is less than both f(ax) and f(cx)], this routine isolates the minimum to a fractional
    precision of about tol using a modification of Brent's method that uses derivatives. The
    abscissa of the minimum is returned as xmin, and the minimum function value is returned
    as min, the returned function value.
    {
        const Int ITMAX=100;
```

```

const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
Bool ok1,ok2;                                Will be used as flags for whether pro-
Doub a,b,d=0.0,d1,d2,du,dv,dw,dx,e=0.0;      posed steps are acceptable or not.
Doub fu,fv,fw,fx,olde,tol1,tol2,u,u1,u2,v,w,x,xm;

```

Comments following will point out only differences from the routine in Brent. Read that routine first.

```

a=(ax < cx ? ax : cx);
b=(ax > cx ? ax : cx);
x=w=v=bx;
fw=fv=fx=funcd(x);
dw=dv=dx=funcd.df(x);
for (Int iter=0;iter<ITMAX;iter++) {
    xm=0.5*(a+b);
    tol1=tol*abs(x)+ZEPS;
    tol2=2.0*tol1;
    if (abs(x-xm) <= (tol2-0.5*(b-a))) {
        fmin=fx;
        return xmin=x;
    }
    if (abs(e) > tol1) {
        d1=2.0*(b-a);
        d2=d1;
        if (dw != dx) d1=(w-x)*dx/(dx-dw);
        if (dv != dx) d2=(v-x)*dx/(dx-dv);
        Initialize these d's to an out-of-bracket
        value.
        Secant method with one point.
        And the other.
        Which of these two estimates of d shall we take? We will insist that they be
        within the bracket, and on the side pointed to by the derivative at x:
        u1=x+d1;
        u2=x+d2;
        ok1 = (a-u1)*(u1-b) > 0.0 && dx*d1 <= 0.0;
        ok2 = (a-u2)*(u2-b) > 0.0 && dx*d2 <= 0.0;
        olde=e;
        e=d;
        Movement on the step before last.
        if (ok1 || ok2) {
            if (ok1 && ok2)
                d=(abs(d1) < abs(d2) ? d1 : d2); the smallest one.
            else if (ok1)
                d=d1;
            else
                d=d2;
            if (abs(d) <= abs(0.5*olde)) {
                u=x+d;
                if (u-a < tol2 || b-u < tol2)
                    d=SIGN(tol1,xm-x);
            } else {
                Bisect, not golden section.
                d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
                Decide which segment by the sign of the derivative.
            }
        } else {
            d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
        }
    } else {
        d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
    }
    if (abs(d) >= tol1) {
        u=x+d;
        fu=funcd(u);
    } else {
        u=x+SIGN(tol1,d);
        fu=funcd(u);
        if (fu > fx) {
            fmin=fx;
            return xmin=x;
        }
    }
}

```

If the minimum step in the downhill direction takes us uphill, then we are done.